

CSCE 649 PHYSICALLY-BASED MODELING · MODULE 01 · LECTURE NOTES

Simulation as an Initial Value Problem

One structure underneath everything in this course.

MODULE OUTCOMES

- Express a physical system as a state vector and a derivative function.
- Reduce a second-order ODE to a first-order system.
- Explain what 'physically based' means and what standard graphics holds it to.
- Identify which quantities a simulation should conserve.
- Set up the measurement harness used for the rest of the course.

1 The structure underneath everything

This course covers particles, springs, constraints, collisions, rigid bodies, deformable solids, cloth, and fluids. They look like eight unrelated subjects. They are one subject.

$$\mathbf{x}' = \mathbf{f}(\mathbf{x}, \mathbf{t}), \quad \mathbf{x}(\mathbf{t}_0) = \mathbf{x}_0$$

A **state vector** $\mathbf{x}$ containing everything needed to describe the system at an instant, and a **derivative function** $\mathbf{f}$ giving its rate of change. With an initial condition this is an **initial value problem**, and a simulator is a loop that advances it.

System	State vector contains	Derivative computes
Particles	Position and velocity of each particle.	Velocity, and force/mass from gravity, drag, and springs.
Cloth	The same — cloth is a particle system with structured connectivity.	The same, with spring forces from the mesh edges.
Rigid bodies	Position, orientation, linear and angular momentum.	Velocity, angular velocity, force, and torque (Module 07).
Deformables	Node positions and velocities.	Internal forces from the strain energy (Module 09).
Fluids (Eulerian)	Velocity and other fields sampled on a grid.	Advection, pressure, and viscosity terms (Module 11).

The loop is identical in every case. Only the contents of $\mathbf{x}$ and the computation of $\mathbf{f}$ change. Recognising this turns the field from a collection of unrelated recipes into one idea with variations, and it is the single most useful thing in this module.

REDUCING SECOND ORDER TO FIRST ORDER

Newton's second law is second order: the acceleration, not the velocity, is what force determines. Integrators are written for first-order systems.

The reduction is mechanical. Stack position and velocity together into a single state vector:

$$\mathbf{x} = [\mathbf{p}, \mathbf{v}] \Rightarrow \mathbf{x}' = [\mathbf{v}, \mathbf{F}(\mathbf{p}, \mathbf{v})/m]$$

The derivative of the position block is the velocity block, which we already have; the derivative of the velocity block is force over mass, which Newton gives us. The system is now first order and any general-purpose integrator applies.

Do this once, in the architecture. Every system in the course then presents the same interface, and swapping integrators to compare them — which this course does in almost every module — becomes a one-line change rather than a rewrite.

```
struct System {
    virtual int  dim() const = 0;
    virtual void get_state(double* x) const = 0;
    virtual void set_state(const double* x) = 0;
    virtual void derivative(const double* x, double t, double* dx) = 0;
};

struct Integrator {
    virtual void step(System& s, double t, double dt) = 0;
};
```

The common mistake is to write `particle.position += particle.velocity * dt` inside a particle class. It works, and it welds the integration scheme to the system so that comparing integrators requires rewriting everything. **Separate state, derivative, and stepping from the first line of code.**

2 What 'physically based' means in graphics

Field	Primary requirement	Acceptable cost	Failure
Engineering	Accuracy , with quantified error bounds. A 2% error in a bridge simulation may be unacceptable.	Hours or days per solve.	Being wrong.
Science	Validity — agreement with experiment, and predictions that can be falsified.	Supercomputer time.	Disagreeing with measurement.
Graphics	Plausibility, stability, controllability, speed — in that order.	Milliseconds to minutes.	Exploding, or being impossible to art-direct.

STABILITY OUTRANKS ACCURACY, AND THAT DRIVES EVERYTHING

A simulation that is 5% wrong and runs reliably for ten thousand frames is **useful**. A simulation that is exact for 179 frames and then produces infinities is **worthless** — and worse than useless, because someone spent the render time.

So this course will repeatedly accept provably less accurate methods in exchange for better stability.

Position-based dynamics (Module 04) is the extreme case: it does not integrate forces at all, its material behaviour depends on the iteration count and the time step, and it is in essentially every game engine, because it cannot explode.

The important thing is to know you are making the trade. The characteristic failure of this field is making it unconsciously, and then being unable to explain why the simulated result does not match a measured one — or, worse, assuming the simulator is right and the measurement is wrong.

ART DIRECTION IS A HARD REQUIREMENT

A director will say the explosion should be bigger, the cape should settle two seconds sooner, and the splash should go to the left of frame.

Physics has no 'go left' parameter. A simulator that cannot be steered will not be used on a production, no matter how physically faithful it is — because the shot has to match the previous one and the storyboard.

So production systems expose controls that are frankly unphysical: per-object gravity, damping multipliers, stiffness that varies across the mesh, forces that act only during a specified frame range, and target shapes the simulation is attracted toward.

Design for this from the beginning. Burying real physical constants inside the solver, with no way to override them per object or per frame, is how you build something that is technically impressive and professionally unusable.

3 Measurement

Quantity	Conserved when	Drift means
Linear momentum	No net external force acts.	Forces are not equal and opposite. Usually a spring applying force to one endpoint and not the other, or a collision impulse applied asymmetrically.
Angular momentum	No net external torque acts.	A force is being applied at the wrong point — at the centre of mass rather than the contact point, typically.
Energy	No damping, friction, or other dissipation.	Almost always the integrator (Module 02). This is the most informative plot in the course.
Volume	The material is incompressible.	A fluid or soft body is leaking — usually a pressure solve that has not converged (Modules 09, 11).

THE ENERGY CURVE IDENTIFIES THE INTEGRATOR

Simulate something with no dissipation — a particle in a circular orbit under an inverse-square force is ideal, because the exact solution is known — and plot total energy against time over ten thousand steps.

Rising steadily: explicit Euler. It injects energy at every step, without exception, and the orbit spirals outward until it escapes.

Oscillating within a fixed band around the true value: a **symplectic** method — symplectic Euler or velocity Verlet. The error is bounded for all time rather than accumulating, which is usually exactly what you want.

Slowly decreasing: RK4 or another high-order non-symplectic scheme. Very accurate over a few steps, mildly dissipative over thousands.

Produce this plot before implementing anything else. Module 02 explains why each curve has the shape it does, and you will refer to this diagnostic in every subsequent module.

3.1 The harness to build first

- **An energy and momentum logger** that runs every step and appends to a CSV. Cheap, and it costs nothing to leave running.
- **A plotting script** that turns that CSV into a figure in one command. You will make hundreds of these plots; friction here means you will stop making them, which is how simulations go wrong unnoticed.
- **Frame stepping, pausing, and rewind in the viewer.** A simulation that can only run forwards at full speed cannot be debugged, because the interesting moment is always one frame before you noticed.
- **Deterministic replay.** See below.
- **Analytic test cases.** Free fall under constant gravity, a single undamped spring, and a circular orbit all have closed-form solutions. Any error in these is unambiguous.

DETERMINISM IS NOT OPTIONAL

Simulation bugs are characteristically intermittent and late: the stack collapses at frame 847, but only sometimes.

If the run is not reproducible, you cannot bisect the failure, cannot verify a fix, and cannot distinguish a fix from luck. You will spend days on something that a reproducible case resolves in an hour.

The usual causes of non-determinism: reading uninitialised memory; iterating over a hash container whose order depends on allocation addresses; parallel reductions that sum in a non-deterministic order (floating-point addition is not associative); and uncontrolled random seeds.

Fixed time steps help enormously. A step size tied to real frame time makes every run different by construction, and makes the simulation's behaviour depend on the machine it runs on. Module 13 covers how to keep a fixed simulation step while rendering at a variable rate.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Baraff & Witkin — Differential Equation Basics (free notes)	The first set of notes in the course pack, and the source of §1's framing. Short and exceptionally clear.
Ten Minute Physics — episodes 1–3	Getting a first simulation running, with working code. Do this before Module 02.
Bargteil & Shinar — An Introduction to Physics-Based Animation	The survey's opening sections cover exactly this module's framing, with a wider view of where the field has gone.
Hairer, Lubich & Wanner — Geometric Numerical Integration (chapter 1 free)	Why symplectic integrators behave as they do. Heavy going, and the first chapter's figures explain the energy curves of §3 better than anything else.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement the System and Integrator interfaces of §1 and a single particle under gravity. Verify against the closed-form solution for free fall.
- 2 Add a spring and verify against the analytic solution for simple harmonic motion. Report the error after 1, 10, and 100 periods.

- 3 Build the energy and momentum logger and the plotting script. This is infrastructure for the whole course — do it properly.
- 4 Simulate a particle in a circular orbit with explicit Euler and plot total energy over 10,000 steps. Report the percentage gained.
- 5 Write down the state vector and derivative function for three systems of your choosing that are *not* in the table in §1 — a pendulum, a rocket losing mass, a population model.
- 6 Make your simulation deterministic and prove it: run the same scenario twice and compare the final state bit for bit.
- 7 Deliberately break determinism by iterating over an `unordered_map` of particles, and show the final states differ.
- 8 Implement frame stepping and rewind in your viewer. You will use them constantly from Module 05 onward.
- 9 Write a page on a simulation you have seen in a film or game, identifying where it was art-directed rather than simulated. The evidence is usually visible once you look.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Write the general form every system in this course takes.
- 2 How is a second-order Newtonian system reduced to first order, and why is that necessary?
- 3 Why should state, derivative, and integrator be separated in the architecture?
- 4 Give the primary requirement of engineering, scientific, and graphics simulation, and the failure mode of each.
- 5 Why does stability outrank accuracy in graphics, and what does that licence?
- 6 Why is art-directability a hard requirement rather than a nice-to-have?
- 7 Name four conserved quantities and say what drift in each one indicates.
- 8 What three shapes can an energy curve take, and what does each one identify?
- 9 Give three causes of non-determinism in a simulator and say why it matters.